LAUREA MAGISTRALE
UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

Dipartimento di **Informatica, Sistemistica e Comunicazione**

Laurea magistrale in **informatica**

Titolo da decidere

Francesco Romeo
Matricola: 885880

Relatrice: **Dott.ssa Elisabetta Fersini**
Co-relatore: **Dottor Marco Lorengian**

Anno Accademico **2025/2026**

Sommario

A one-page "technical" abstract that summarizes your research work

Indice

Introduzione **i**

1 Stato dell'Arte e Background Tecnologico **1**
1.1 Architetture Transformer . . . 1
1.1.1 Dalla traduzione automatica al modello universale . . . 1
1.1.2 Il meccanismo di Self-Attention . . . 1
1.1.3 Architettura encoder-decoder e varianti decoder-only . . . 2
1.1.4 Limiti nel contesto zero-shot e motivazione del RAG . . . 2
1.2 Large Language Models: panorama e analisi comparativa . . . 3
1.2.1 Modelli proprietari . . . 3
1.2.2 Modelli open source per uso on-premise . . . 3
1.2.3 Quantizzazione e inferenza on-premise . . . 4
1.2.4 Ollama: server di inferenza locale . . . 4
1.3 Il Paradigma Retrieval-Augmented Generation (RAG) . . . 4
1.3.1 Motivazione e definizione . . . 4
1.3.2 Architettura del sistema RAG . . . 5
1.3.3 Modelli di embedding . . . 6
1.3.4 Database vettoriali . . . 6
1.3.5 Tecniche di adattamento al dominio . . . 6
1.4 Framework di orchestrazione: Haystack . . . 7
1.5 Sintesi e posizionamento del lavoro . . . 8

Conclusioni e Sviluppi Futuri **9**

A Annotazioni e Convenzioni **10**

B Codice Utilizzato **11**

Bibliografia **12**

Ringraziamenti **13**

Introduzione

* Contesto: Diffusione dei Large Language Models (LLM) e le sfide dell'adozione in ambito corporate (privacy, allucinazioni, aggiornamento dei dati). —¿ giustificare le affermazioni con citazioni e dati. * Il Problema: Gestione della conoscenza non strutturata e valorizzazione del materiale formativo (Gilde) in Intré. —¿ descrizione dell'azienda e dello use-case. * Obiettivo della tesi: Progettazione e sviluppo di un sistema RAG (Retrieval-Augmented Generation) con focus sull'ottimizzazione dell'inferenza e dell'accuratezza nel dominio specifico. * Estrema sintesi dei risultati ottenuti * NOTA: essere esplicativi sotto tutti i punti di vista, deve essere una versione sintetica della testi - un riassunto - con l'aggiunta delle motivazioni del progetto.

Capitolo 1

Stato dell'Arte e Background Tecnologico

Il presente capitolo fornisce le fondamenta teoriche e tecnologiche necessarie alla comprensione del sistema progettato nei capitoli successivi. Vengono dapprima introdotte le architetture Transformer, su cui si basano tutti i modelli linguistici di larga scala (*Large Language Models*, LLM) considerati nel progetto; si analizza successivamente lo spazio dei modelli aperti e proprietari, con particolare attenzione alle soluzioni adatte all'esecuzione on-premise; si illustra infine il paradigma *Retrieval-Augmented Generation* (RAG) nella sua architettura complessiva e nelle tecniche di adattamento al dominio.

1.1 Architetture Transformer

1.1.1 Dalla traduzione automatica al modello universale

Il modello Transformer, introdotto da Vaswani et al. nel 2017 [25], ha ridefinito lo stato dell'arte nel trattamento del linguaggio naturale (*Natural Language Processing*, NLP). Prima di tale contributo, i sistemi dominanti erano basati su reti neurali ricorrenti (*Recurrent Neural Networks*, RNN) e, più specificamente, su architetture LSTM (*Long Short-Term Memory*) [9], che elaboravano le sequenze in modo sequenziale, accumulando un vettore di stato nascosto (*hidden state*) ad ogni passo temporale. Tale formulazione introduceva due limitazioni strutturali: l'impossibilità di parallelizzare il calcolo durante l'addestramento e la difficoltà di mantenere dipendenze a lungo raggio in sequenze di lunghezza elevata, problema noto come *vanishing gradient* [1].

Il Transformer risolve entrambi i problemi abbandonando la ricorrenza in favore di un meccanismo di *self-attention* globale, che consente a ciascun elemento di una sequenza di relazionarsi direttamente con tutti gli altri, indipendentemente dalla distanza posizionale.

1.1.2 Il meccanismo di Self-Attention

Il nucleo computazionale del Transformer è il meccanismo di *attention* scalato. Data una sequenza di token rappresentati come vettori di dimensione d_{model}, vengono calcolate tre proiezioni lineari

per ogni elemento: la query $\mathbf{Q}$, la chiave $\mathbf{K}$ e il valore $\mathbf{V}$. Il punteggio di attenzione tra due elementi è definito come:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{1.1}$$

dove d_k è la dimensione dei vettori di chiave. Il fattore di scala $1/\sqrt{d_k}$ previene la saturazione della funzione softmax in presenza di prodotti scalari di grande magnitudine, che produrrebbe gradienti numericamente instabili [25].

Il meccanismo viene esteso a più "teste" parallele, ciascuna delle quali apprende una proiezione diversa dello stesso spazio di input: si parla di *Multi-Head Attention* (MHA). La concatenazione degli output di h teste permette al modello di catturare simultaneamente relazioni di tipo sintattico, semantico e posizionale:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\,\mathbf{W}^O \tag{1.2}$$

con $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$.

La posizione dei token nella sequenza, non codificata implicitamente dalla struttura ricorrente, viene iniettata tramite *positional encoding*: vettori sinusoidali sommati agli embedding di input, che forniscono al modello informazioni sull'ordine relativo degli elementi.

1.1.3 Architettura encoder-decoder e varianti decoder-only

L'architettura originale del Transformer è di tipo encoder-decoder: un encoder trasforma la sequenza di input in una rappresentazione contestuale densa, mentre un decoder la utilizza — attraverso un meccanismo di *cross-attention* — per generare la sequenza di output. Questa struttura è particolarmente adatta a task *sequence-to-sequence* come la traduzione automatica [25].

I modelli linguistici generativi contemporanei adottano prevalentemente un'architettura *decoder-only*, nella quale l'attenzione è applicata in modo *causale* (o *masked*): ogni token può attendere esclusivamente ai token che lo precedono nella sequenza. Questo vincolo è necessario per l'addestramento autoregressivo, dove il modello apprende a predire il token successivo dato il contesto precedente.

1.1.4 Limiti nel contesto zero-shot e motivazione del RAG

Nonostante l'efficacia dei modelli Transformer pre-addestrati su corpora di grandi dimensioni, la loro applicazione in contesti aziendali specifici presenta limitazioni strutturali non trascurabili.

Il primo limite è la **conoscenza statica**: i parametri del modello codificano la conoscenza del corpus di addestramento, il cui orizzonte temporale è fisso (*knowledge cutoff*). Il modello non ha accesso a informazioni aggiornate o a documenti interni non presenti durante l'addestramento.

Il secondo limite è il fenomeno delle **allucinazioni**: quando interrogato su informazioni assenti nella propria distribuzione di addestramento, il modello tende a generare testo plausibile ma fattualmente errato [11]. Questo comportamento è particolarmente critico in applicazioni aziendali dove l'accuratezza fattuale è un requisito non negoziabile.

Il terzo limite riguarda la **privacy dei dati**: addestrare o eseguire *fine-tuning* su un modello di un fornitore esterno per incorporare conoscenza aziendale riservata introduce rischi di esposizione dei dati verso infrastrutture di terze parti.

Questi tre fattori motivano l'adozione del paradigma RAG, discusso nella Sezione 1.3, come alternativa pragmatica al fine-tuning per l'adattamento al dominio.

1.2 Large Language Models: panorama e analisi comparativa

L'ecosistema degli LLM si è diversificato rapidamente a partire dal 2022, con una chiara polarizzazione tra modelli proprietari, accessibili esclusivamente tramite API, e modelli open source, distribuiti con pesi pubblicamente scaricabili e utilizzabili on-premise.

1.2.1 Modelli proprietari

I modelli proprietari rappresentano lo stato dell'arte in termini di prestazioni su benchmark generali, ma presentano vincoli rilevanti per l'adozione aziendale in contesti con requisiti stringenti di riservatezza.

GPT-4 / GPT-4o (OpenAI). La famiglia GPT-4 [20] è attualmente la famiglia di riferimento in termini di ragionamento complesso e capacità zero-shot su task eterogenei. GPT-4o introduce ottimizzazioni significative per la latenza e il costo per token rispetto ai predecessori. L'accesso avviene esclusivamente tramite API cloud, con trasferimento dei dati verso infrastrutture OpenAI: questo aspetto rende la soluzione inadatta a contesti in cui i dati non possono lasciare il perimetro aziendale.

Gemini (Google DeepMind). La famiglia Gemini [7] è caratterizzata da capacità multimodali native e da una context window particolarmente ampia (fino a 2M token per Gemini 1.5 Pro). Come GPT-4, è disponibile unicamente in modalità cloud-hosted, tramite Google Cloud Vertex AI o l'API pubblica.

Entrambi i modelli sono pertanto **esclusi** dall'implementazione descritta in questa tesi, che impone come vincolo architetturale l'esecuzione interamente on-premise.

1.2.2 Modelli open source per uso on-premise

La disponibilità di modelli open source con prestazioni competitive ha reso praticabile l'adozione di LLM in contesti aziendali con vincoli infrastrutturali e di privacy. Di seguito si analizzano i modelli rilevanti per il progetto.

Llama 3 / Llama 3.1 (Meta AI). La famiglia Llama 3 [24, 16] rappresenta un punto di riferimento per i modelli open source. Llama 3.1, rilasciata nell'estate del 2024, introduce dimensioni da 8B, 70B e 405B parametri, con quest'ultima che compete direttamente con GPT-4 su diversi benchmark. La context window di 128k token di Llama 3.1 la rende adatta a task di reasoning su documenti lunghi. I pesi sono distribuiti con una licenza personalizzata che consente l'uso commerciale per la maggior parte degli scenari.

Mistral / Mixtral (Mistral AI). Mistral 7B [12] ha dimostrato che modelli di dimensioni contenute, opportunamente addestrati, possono superare modelli significativamente più grandi (ad esempio LLaMA 2 13B) su benchmark di ragionamento e comprensione del testo. Mixtral 8x7B [13] introduce l'architettura *Mixture of Experts* (MoE), in cui l'inferenza attiva soltanto un sottoinsieme degli esperti per ogni token, mantenendo qualità comparabile a un modello denso di 70B con un

costo computazionale corrispondente a 12B parametri attivi. L'intera famiglia Mistral è distribuita sotto licenza Apache 2.0.

Qwen2 (Alibaba Cloud). La famiglia Qwen2 [22] comprende modelli da 0.5B a 72B parametri, con copertura multilingue estesa che include lingue non latine. Qwen2 si distingue per il favorevole rapporto qualità/dimensioni nelle fasce inferiori (0.5B – 1.5B), che lo rendono praticabile per inferenza su hardware con GPU limitate o su CPU. Come discusso nel Capitolo **??**, il modello `qwen2:1.5b` è stato selezionato come modello di chat nell'implementazione del sistema per il suo bilanciamento tra qualità delle risposte e velocità di inferenza su hardware CPU.

1.2.3 Quantizzazione e inferenza on-premise

Un aspetto centrale per l'esecuzione locale di LLM è la **quantizzazione**: la riduzione della precisione numerica dei pesi del modello da 32-bit floating point (FP32) o 16-bit (FP16/BF16) a rappresentazioni a bassa precisione (8-bit, 4-bit o inferiori). Tecniche come GPTQ [6] e GGUF/llama.cpp [8] hanno reso accessibile l'esecuzione di modelli da 7B–13B parametri su hardware consumer, con una riduzione dell'occupazione di memoria dal 60% all'80% a seconda del bit-width scelto, a fronte di una degradazione delle prestazioni tipicamente contenuta entro 1-3 punti percentuali sui benchmark principali [5].

L'impatto della quantizzazione sulla qualità della generazione in contesti domain-specific viene analizzato nel Capitolo **??**.

1.2.4 Ollama: server di inferenza locale

Per il progetto in esame, l'infrastruttura di inferenza è gestita tramite **Ollama** [19], un server open source che espone i modelli GGUF tramite un'API REST compatibile con la specifica OpenAI Chat Completions. Ollama semplifica drasticamente il ciclo di vita dei modelli locali: l'installazione avviene tramite singolo comando (`ollama pull <modello>`), la gestione del contesto e del formato di prompt è automatizzata, e il server è distribuibile tramite un container Docker ufficiale. Nel contesto di questo progetto, Ollama serve sia il modello di chat (`qwen2:1.5b`) sia il modello di embedding (`nomic-embed-text`), il cui ruolo è descritto nella sezione successiva.

1.3 Il Paradigma Retrieval-Augmented Generation (RAG)

1.3.1 Motivazione e definizione

Il paradigma *Retrieval-Augmented Generation* (RAG), formalizzato da Lewis et al. [14], nasce per superare la limitazione fondamentale dei LLM come sistemi a conoscenza chiusa (*closed-book*): la conoscenza del modello è interamente codificata nei suoi parametri e non può essere aggiornata senza un nuovo ciclo di addestramento o fine-tuning.

In un sistema RAG, il processo di generazione è arricchito da una fase di *retrieval* dinamico: prima di invocare l'LLM, viene eseguita una ricerca in una base di conoscenza esterna per recuperare i documenti pertinenti alla query dell'utente. Tali documenti vengono iniettati nel contesto del prompt (*prompt augmentation*), permettendo al modello di fondare la propria risposta su informazioni aggiornate e verificabili.

Il vantaggio rispetto al fine-tuning è duplice: non richiede un nuovo ciclo di addestramento ogni volta che la base di conoscenza viene aggiornata, e consente la tracciabilità delle fonti, riducendo il rischio di allucinazioni [23].

1.3.2 Architettura del sistema RAG

Un sistema RAG si articola in due macro-pipeline distinte: la pipeline di **ingestione** (*ingestion pipeline*), eseguita offline, e la pipeline di **inferenza** (*query pipeline*), eseguita in tempo reale per ogni richiesta dell'utente.

Pipeline di ingestione

La pipeline di ingestione è responsabile della trasformazione dei documenti grezzi in vettori numerici persistenti in un database vettoriale. Le fasi principali sono le seguenti.

Pre-processing e chunking. I documenti sorgente vengono suddivisi in unità testuali (*chunk*) di dimensione controllata. La scelta della strategia di chunking impatta direttamente sulla qualità del retrieval: chunk troppo piccoli perdono il contesto locale, chunk troppo grandi superano la finestra di contesto del modello di embedding e diluiscono il segnale semantico [3]. Le strategie più comuni includono la divisione a finestra fissa con overlap, la divisione per struttura del documento (paragrafi, sezioni) e la divisione semantica basata sulla coerenza del testo.

Generazione degli embedding. Ogni chunk viene trasformato in un vettore denso di dimensione fissa da un modello di embedding. Un modello di embedding $f : \mathcal{T} \to \mathbb{R}^d$ mappa uno spazio testuale $\mathcal{T}$ in uno spazio vettoriale $\mathbb{R}^d$ tale che documenti semanticamente simili producano vettori vicini secondo una metrica di distanza — tipicamente la similarità coseno:

$$\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|} \tag{1.3}$$

Indicizzazione nel database vettoriale. I vettori prodotti vengono persistiti in un database vettoriale ottimizzato per la ricerca per similarità (*Approximate Nearest Neighbor*, ANN). Nel progetto descritto in questa tesi è stato adottato **Qdrant** [21], un database vettoriale scritto in Rust che implementa l'algoritmo HNSW (*Hierarchical Navigable Small World*) [15] per l'indicizzazione. La motivazione della scelta è approfondita nel Capitolo **??**.

Pipeline di inferenza

La pipeline di inferenza elabora ogni query utente in tre passi successivi.

Embedding della query. La query dell'utente viene trasformata nel medesimo spazio vettoriale dei documenti indicizzati, utilizzando lo stesso modello di embedding.

Retrieval. Viene eseguita una ricerca ANN nel database vettoriale per recuperare i k documenti più simili alla query (*top-k retrieval*). La ricerca può essere filtrata per attributi strutturali dei documenti (*metadata filtering*), permettendo di restringere lo spazio di ricerca a sottoinsiemi semanticamente coerenti (ad esempio, documenti appartenenti a una specifica epoca temporale nel sistema i3Guild).

Augmented generation. Il contesto recuperato viene concatenato alla query nel prompt inviato all'LLM, secondo un template strutturato. Il modello genera la risposta basandosi sia sulla propria conoscenza parametrica sia sulle informazioni esterne fornite.

Figura 1.1: Schema della pipeline RAG: la freccia superiore rappresenta la fase di ingestione offline; la freccia inferiore rappresenta la pipeline di inferenza in tempo reale.

1.3.3 Modelli di embedding

La scelta del modello di embedding è una decisione architetturale con impatto diretto sulla qualità del retrieval. Nel contesto di questo progetto, il vincolo di esecuzione on-premise restringe la scelta a modelli distribuibili localmente tramite Ollama. La valutazione comparativa è stata condotta su quattro candidati — `nomic-embed-text`, `mxbai-embed-large`, `all-minilm` e `multilingual-e5-large` — secondo criteri di qualità in lingua italiana, occupazione di memoria, compatibilità con Ollama e punteggi sul benchmark MTEB (*Massive Text Embedding Benchmark*) [17]. L'analisi completa e la motivazione della scelta finale di `nomic-embed-text` [18] sono presentate nel Capitolo **??**.

1.3.4 Database vettoriali

Un database vettoriale è un sistema di storage specializzato per vettori densi ad alta dimensionalità, progettato per rispondere efficientemente a query di tipo *k-nearest neighbor* (kNN) nello spazio vettoriale. A differenza dei database relazionali, dove le query sono esatte, la ricerca vettoriale è per natura approssimata: si ricercano i vettori più vicini secondo una metrica di distanza, accettando un trade-off tra precisione (*recall*) e velocità (*throughput*).

L'algoritmo HNSW costruisce un grafo gerarchico multi-livello sullo spazio vettoriale, permettendo una navigazione greedy dalla radice verso i nodi foglia più vicini alla query. La complessità di ricerca è $O(\log n)$ rispetto al numero di punti n, contro $O(n)$ della ricerca lineare esatta [15]. Questo garantisce la scalabilità del sistema anche in presenza di corpora di grandi dimensioni.

1.3.5 Tecniche di adattamento al dominio

L'adattamento di un LLM pre-addestrato a un dominio specifico può avvenire secondo due approcci fondamentali: l'adattamento parametrico tramite *fine-tuning* e l'adattamento non parametrico tramite RAG e *prompt engineering*. Questa sezione analizza le tecniche di fine-tuning con parametri ridotti, che costituiscono il riferimento teorico per il confronto discusso nel Capitolo **??**.

Fine-tuning con adattatori: LoRA

Il *fine-tuning* completo di un LLM da miliardi di parametri richiede risorse computazionali proibitive per la maggior parte degli scenari aziendali. Per ovviare a questa limitazione, Hu et al. [10] hanno proposto *Low-Rank Adaptation* (LoRA), una tecnica che vincola gli aggiornamenti dei pesi del modello a matrici di rango basso.

Dato un layer lineare con matrice dei pesi $\mathbf{W}_0 \in \mathbb{R}^{m \times n}$, LoRA introduce una decomposizione degli aggiornamenti nella forma:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{B}\mathbf{A} \tag{1.4}$$

dove $\mathbf{A} \in \mathbb{R}^{r \times n}$ e $\mathbf{B} \in \mathbb{R}^{m \times r}$ con $r \ll \min(m, n)$. I pesi originali $\mathbf{W}_0$ rimangono congelati durante l'addestramento; solo i parametri $\mathbf{A}$ e $\mathbf{B}$ vengono ottimizzati. Il numero di parametri addestrabili

si riduce da $m \times n$ a $r \times (m + n)$: con $r = 8$ e $m = n = 4096$, il risparmio è di circa due ordini di grandezza.

QLoRA: fine-tuning a 4-bit

Quantized LoRA (QLoRA), introdotto da Dettmers et al. [5], estende LoRA combinando la quantizzazione 4-bit del modello base con gli adattatori LoRA a precisione piena. I pesi del modello pre-addestrato vengono quantizzati in formato NF4 (*NormalFloat4*), una rappresentazione a 4-bit ottimizzata per pesi normalmente distribuiti; durante il forward pass, i pesi vengono de-quantizzati dinamicamente a BF16 per il calcolo, mentre gli adattatori LoRA operano sempre in BF16.

Questa combinazione permette il fine-tuning di modelli da 65B parametri su hardware con una singola GPU da 48 GB, mantenendo prestazioni comparabili al fine-tuning full-precision [5]. La Tabella 1.1 riassume i principali trade-off tra le due varianti.

Tabella 1.1: Confronto tra LoRA e QLoRA in termini di requisiti hardware e qualità attesa.

Caratteristica	**LoRA**	**QLoRA**
Precisione pesi base	FP16/BF16	NF4 (4-bit)
Precisione adattatori	FP16/BF16	BF16
Memoria GPU (modello 7B)	~14 GB	~4 GB
Overhead di de-quantizzazione	assente	presente
Degradazione qualitativa	trascurabile	$< 1\%$ su benchmark principali

Prompt engineering e inference-time intervention

In alternativa al fine-tuning parametrico, è possibile adattare il comportamento di un LLM intervenendo sulla formulazione dell'input (*prompt engineering*) senza modificare i pesi del modello. Tecniche come *few-shot prompting* [2] e *chain-of-thought prompting* [26] permettono di guidare il modello verso risposte più accurate e strutturate semplicemente arricchendo il contesto del prompt con esempi o istruzioni di ragionamento intermedie.

Nel sistema descritto in questa tesi, il prompt engineering è impiegato a due livelli: nella definizione del *system prompt* che circoscrive il dominio di risposta del chatbot, e nell'iniezione strutturata del contesto recuperato dal sistema RAG. Un'analisi delle strategie adottate e dei loro effetti sulla qualità delle risposte è presentata nel Capitolo **??**.

1.4 Framework di orchestrazione: Haystack

La costruzione di sistemi RAG richiede l'integrazione di componenti eterogenei — modelli di embedding, database vettoriali, LLM, pipeline di pre-processing — attraverso interfacce coerenti e componibili. **Haystack** [4], sviluppato da deepset, è un framework Python open source specificatamente progettato per questo scopo.

Nella versione 2.x adottata in questo progetto, Haystack introduce un modello a *pipeline*: ogni pipeline è un grafo diretto aciclico (DAG) di componenti con interfacce tipizzate, che possono essere collegati in modo dichiarativo. Sono disponibili integrazioni native per Qdrant (`QdrantDocumentStore`),

Ollama (`OllamaDocumentEmbedder`, `OllamaTextEmbedder`, `OllamaGenerator`) e le principali operazioni di pre-processing (`DocumentCleaner`, `DocumentSplitter`).

Il dettaglio implementativo della pipeline di ingestione e di retrieval Haystack è descritto nel Capitolo **??**.

1.5 Sintesi e posizionamento del lavoro

Il presente capitolo ha fornito il quadro teorico di riferimento su cui si fonda il sistema descritto nei capitoli successivi. Le architetture Transformer costituiscono il substrato computazionale comune a tutti i componenti di generazione e di embedding; i limiti intrinseci di tali modelli in contesti a conoscenza chiusa motivano l'adozione del paradigma RAG. La scelta di operare esclusivamente con modelli e infrastrutture open source e on-premise — Ollama, Qdrant, Haystack — risponde ai vincoli di privacy e sostenibilità computazionale identificati nell'analisi dei requisiti. Le tecniche di fine-tuning con basso numero di parametri (LoRA, QLoRA) costituiscono il termine di confronto teorico per la valutazione delle strategie di adattamento al dominio discusse nel Capitolo **??**.

Conclusioni e Sviluppi Futuri

* Sintesi del lavoro: Efficacia del sistema RAG nell'ecosistema aziendale. * Contributi originali: Risultati dell'analisi metodologica sull'attenzione e l'inferenza. * Limiti e lavori futuri: Agenti multi-modali, miglioramento continuo della base di conoscenza.

Appendice A

Annotazioni e Convenzioni

Text

Appendice B

Codice Utilizzato

Text

Bibliografia

[1] Yoshua Bengio, Patrice Simard e Paolo Frasconi. "Learning Long-Term Dependencies with Gradient Descent is Difficult". In: vol. 5. 2. 1994, pp. 157–166.

[2] Tom Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.

[3] Jiawei Chen et al. "Benchmarking Large Language Models in Retrieval-Augmented Generation". In: 2023.

[4] deepset GmbH. *Haystack: Open Source NLP Framework*. 2023. URL: `https://haystack.deepset.ai`.

[5] Tim Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems* 36 (2023).

[6] Elias Frantar et al. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers". In: *International Conference on Learning Representations*. 2023.

[7] Gemini Team e Google DeepMind. "Gemini: A Family of Highly Capable Multimodal Models". In: *arXiv preprint arXiv:2312.11805* (2023).

[8] Georgi Gerganov. *llama.cpp*. 2023. URL: `https://github.com/ggerganov/llama.cpp`.

[9] Sepp Hochreiter e Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[10] Edward J. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: (2022).

[11] Ziwei Ji et al. "Survey of Hallucination in Natural Language Generation". In: *ACM Computing Surveys* 55.12 (2023), pp. 1–38.

[12] Albert Q. Jiang et al. "Mistral 7B". In: *arXiv preprint arXiv:2310.06825* (2023).

[13] Albert Q. Jiang et al. "Mixtral of Experts". In: *arXiv preprint arXiv:2401.04088* (2024).

[14] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.

[15] Yury A. Malkov e Dmitry A. Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2018), pp. 824–836.

[16] Meta AI. *Introducing Meta Llama 3*. 2024. URL: `https://ai.meta.com/blog/meta-llama-3/`.

[17] Niklas Muennighoff et al. "MTEB: Massive Text Embedding Benchmark". In: *arXiv preprint arXiv:2210.07316* (2022).

[18] Zach Nussbaum et al. "Nomic Embed: Training a Reproducible Long Context Text Embedder". In: *arXiv preprint arXiv:2402.01613* (2024).

[19] Ollama Contributors. *Ollama: Get Up and Running With Large Language Models Locally*. 2023. URL: `https://ollama.com`.

[20] OpenAI. *GPT-4 Technical Report*. Rapp. tecn. OpenAI, 2023. URL: `https://openai.com/research/gpt-4`.

[21] Qdrant Contributors. *Qdrant: Vector Database for the Next Generation of AI Applications*. 2023. URL: `https://qdrant.tech`.

[22] Qwen Team e Alibaba Cloud. *Qwen2 Technical Report*. 2024.

[23] Kurt Shuster et al. "Retrieval Augmentation Reduces Hallucination in Conversation". In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021.

[24] Hugo Touvron et al. "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971* (2023).

[25] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems* 30 (2017).

[26] Jason Wei et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.